

PROJECT DEFINITION & DESIGN PLAN

CareSchedule

A focused medical appointment scheduler

Flask • SQLAlchemy • SQLite • 5 weeks

[Mary Grace Nazareno] | [582-32W-VA] | July 30, 2026



The client needs one reliable scheduling system

Manual communication makes availability unclear and increases the risk of overlapping appointment requests.

- Open times should be visible
- Bookings must persist
- Double booking must be prevented
- Both users need clear feedback

01

Primary problem

Patients cannot easily see open times, while doctors need control over their own schedules.

One workflow serves two clear user roles



Patient

Goal: find and book a suitable appointment without repeated calls.

PRIMARY USER



Doctor

Goal: publish availability and review confirmed bookings.

SECONDARY USER

The five-week scope centers on one complete workflow

ORIGINAL BRIEF

Patients + doctors + clinics + administrators
Medical documents + billing + chat
Reminders + maps + bilingual support

SELECTED MVP



Doctor availability
→ Patient booking

Future improvements remain documented, but they will not delay authentication, validation, database persistence, or the main booking workflow.

Must-have features protect the core experience

- Patient and doctor registration, login, and logout
- Role-based dashboards and protected actions
- Doctor creates and deletes available time slots
- Patient views, books, and cancels appointments
- Server validation prevents invalid or duplicate bookings
- Responsive forms with success, error, and empty states

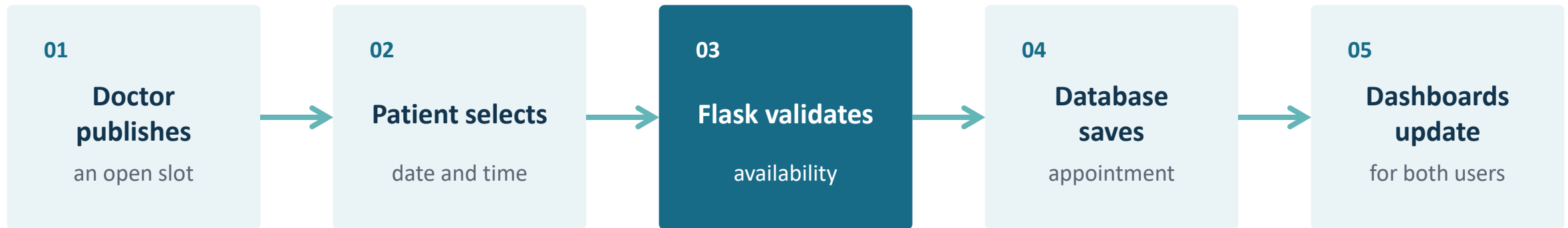
Only after core

- Specialty/date filters
- Edit availability
- Status filters
- English/French UI

Use Case Diagram



The booking workflow connects interface to database



This is the required vertical slice: form → validation → Flask route → database → updated interface.

The Figma plan covers desktop, mobile, and interface states

Login / Register

roles + validation

Patient dashboard

upcoming + empty

Find appointment

filters + results

Book appointment

error + success

Doctor dashboard

bookings + summary

Manage availability

conflict + success

The design prioritizes clarity, consistency, and accessibility

01

Clarity

Available slots are the visual focus; medical details stay minimal.

02

Consistency

Patients and doctors reuse navigation, forms, cards, badges, and alerts.

03

Accessibility

Visible labels, keyboard focus, strong contrast, and non-colour status cues.



Teal



Navy



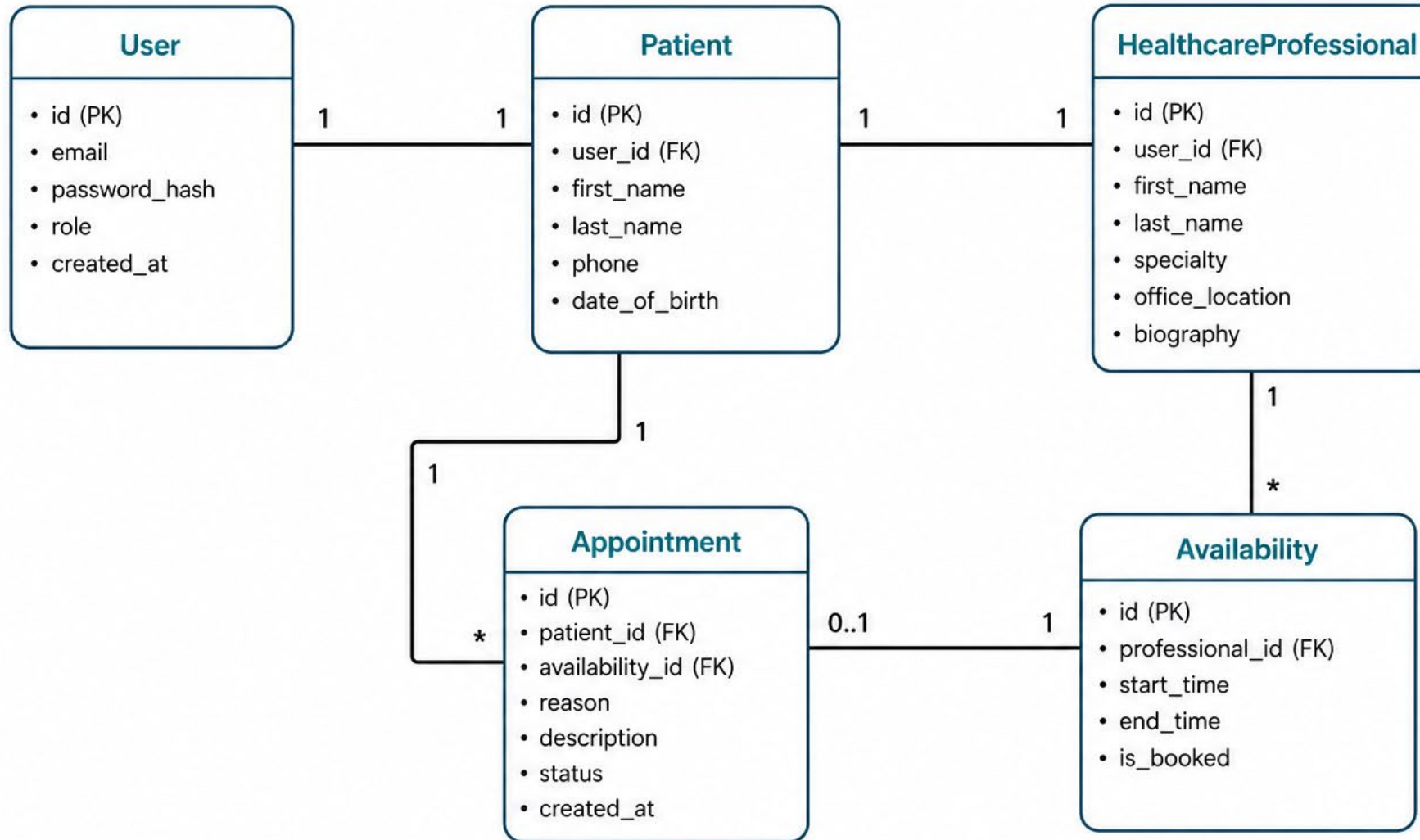
Off-white



Success



Error



Relationships:

User→Patient (1:1)

User→HealthcareProfessional (1:1)

HealthcareProfessional→Availability
(1:many)

Patient→Appointment
(1:many)

Availability→Appointment
(1:0..1)

PK = Primary Key

FK = Foreign Key

Eight routes keep the backend plan understandable

Authentication & navigation

GET/POST /register

GET/POST /login

POST /logout

GET /dashboard

Availability & appointments

GET/POST /availability

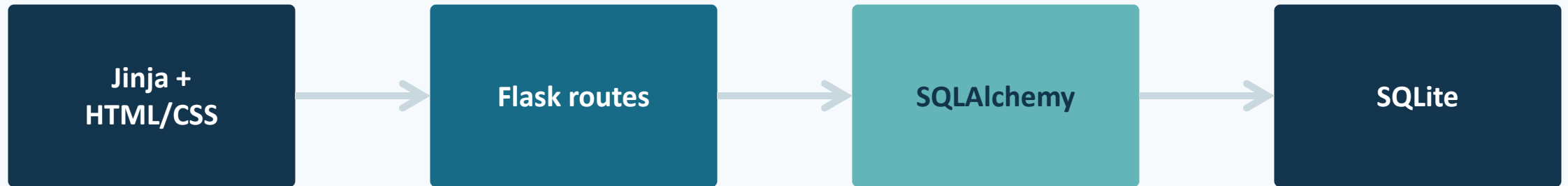
POST /availability/<id>/delete

GET/POST /appointments

POST /appointments/<id>/cancel

Every database change includes validation, ownership checks, and user feedback.

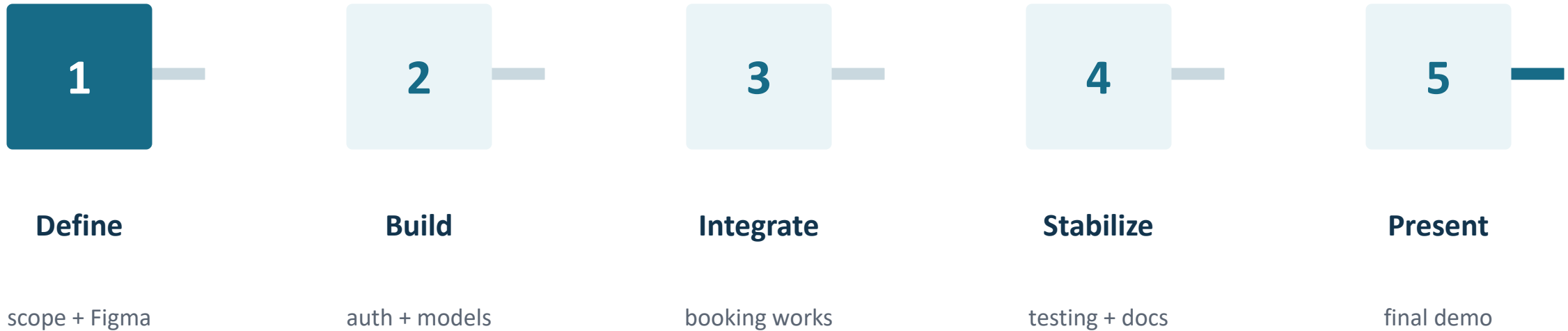
Track A keeps the architecture direct and achievable



JavaScript enhances selected interactions; it does not replace Flask-rendered pages.

CLEAR SEPARATION OF CONCERNS

Each week ends with testable evidence



Largest risk: scope growth. Control: freeze must-haves after Week 1.

The scope is defined. The next step is evidence.

- 01 Finish desktop + mobile Figma frames
- 02 Add Figma and GitHub links
- 03 Show initial commits
- 04 Rehearse the workflow and scope decision

Privacy rule
Use fictional
accounts and
fictional medical
information only.

Never commit
secrets, passwords,
real health
numbers, or
medical files.

A complete scheduling workflow is the measure of success.